



**WYDZIAŁ FIZYKI
i INFORMATYKI STOSOWANEJ**
Uniwersytet Łódzki

Maciej Pracucik

Kierunek: informatyka

Specjalność: informatyka stosowana

Ścieżka dydaktyczna: systemy mobilne

Numer albumu: 410731

Wykorzystanie sieci typu GAN na potrzeby generowania gestów rąk.

Praca magisterska

wykonana pod kierunkiem
dr Krzysztof Podlaski
w Katedrze Systemów Inteligentnych
WFiIS UŁ

Łódź 2025

Spis treści

1	Wprowadzenie	3
1.1	Problematyka	3
1.2	Cel i zakres pracy	3
1.3	Struktura pracy	3
2	Sieci GAN, analiza konkurencji i techniczne aspekty realizacji	4
2.1	Sieci neuronowe i AI	4
2.2	Budowa sieci neuronowej	5
2.3	Pojęcia związane z sieciami neuronowymi	6
2.4	Sieci typu GAN	7
2.5	GestureGAN	8
2.6	PoseGAN	10
3	Narzędzia i technologie wybrane do realizacji projektu	11
3.1	Python	11
3.2	PyTorch	12
3.3	MediaPipe	12
3.4	OpenCV	12
4	Proces tworzenia projektu	13
4.1	Dobór zbioru danych	13
4.2	Wybór architektury sieci	15
4.3	Preprocesowanie danych	16
4.4	Model sieci	19
4.5	Ścieżka uczenia sieci i animacja	22
4.6	Animacja	23
5	Wyniki i dyskusja	23
5.1	CycleGAN	23
5.2	Pix2Pix	27
6	Podsumowanie	33
	Bibliografia	36

1 Wprowadzenie

1.1 Problematyka

W dzisiejszych czasach bardzo modnym tematem jest sztuczna inteligencja, która zaczyna się wkradać w każdy aspekt naszego życia. Możemy ją spotkać w formie chatów, podpowiedzi do pisanego kodu, asystentów internetowych czy systemów rozpoznawania mowy. Każda firma żeby zaistnieć i pozostać istotną inwestuje w tę część technologii. Jedną z gałęzi rozwoju jest generowanie zdjęć, czy filmów. AI już jest w stanie wygenerować przeróżne obrazy, jednakże to z czym AI radzi sobie najgorzej są ręce.[1]

W niniejszej pracy skupia się na stworzeniu sieci typu GAN, która pozwoli na generowanie obrazów rąk, w jak najlepszej jakości. Dodatkowo sieć ma za zadanie nauczyć się, żeby modyfikować istniejące zdjęcia i nadawać im zupełnie inny gest, przy zachowaniu jak najlepszej jakości i realizmu. Szczególnie ten drugi aspekt pozostaje dla sztucznej inteligencji problematyczny. Myślę, że każdy z nas spotkał się ze zdjęciami, które dosłownie wyglądają, jak żywe, ale to co najczęściej zdradza, że jednak to AI maczało w nim palce są ręce. Za długie palce, dziwne ich ułożenie, ilość, czy nawet totalnie odrealniony wygląd. Przeróżne firmy, jak i naukowcy stale ulepszają sieci, i rozwiązania, żeby i to przestało być problemem. Niniejsza praca również podejmuje się tego niełatwego zadania.

1.2 Cel i zakres pracy

Celem niniejszej pracy jest stworzenie sieci neuronowej typu GAN, która pozwoli na generowanie obrazów gestów rąk, w jak najlepszej jakości.

Docelowo również, wygenerowane zdjęcia będą wykorzystywane do stworzenia animacji przechodzenia z jednego gestu w inny.

1.3 Struktura pracy

Pierwszy rozdział przybliży to czym są sieci neuronowe, a dokładniej typu GAN, jakie są analogiczne rozwiązania, oraz o samym generowaniu zdjęć. Następny opowie jakie narzędzia, biblioteki i technologie zostały wykorzystane do realizacji projektu. Trzeci zaś mówi o tym jak wyglądał proces tworzenia projektu. Co po kolei zostało zrobione, jakie po drodze wystąpiły komplikacje, oraz jak zostały rozwiązane i finalnie jak wygląda projekt. Przedostatni rozdział to przedstawienie wyników, rezultatów realizowanego projektu,

analiza i omówienie ich. Ostatni rozdział zawiera wnioski końcowe i podsumowanie.

2 Sieci GAN, analiza konkurencji i techniczne aspekty realizacji

2.1 Sieci neuronowe i AI

Człowiek od samego początku swojego istnienia jest istotą niesamowicie ciekawą. To ona sprawia, że w głowie ludzi pojawiają się pytania, a co jeśli? A co jeśli to co wiemy to jest tylko część prawdy? Co jeśli jest coś więcej? To dzięki zadawaniu sobie przeróżnych pytań przez różne osoby, najczęściej przez największe umysły jakie chodziły po tej ziemi tak dużo udało nam się osiągnąć. Poczynając od wynalezienia koła, silniki parowe, elektryczność, loty w kosmos aż po internet. Niektóre z tych wynalazków łączy kolejny aspekt, to że człowiek chce sobie ułatwiać codzienne zadania. Żeby jak najbardziej zwiększyć swoją produktywność, żeby codziennie czynności nie wchodziły w drogę, albo żeby je po prostu ułatwić czy przyspieszyć.

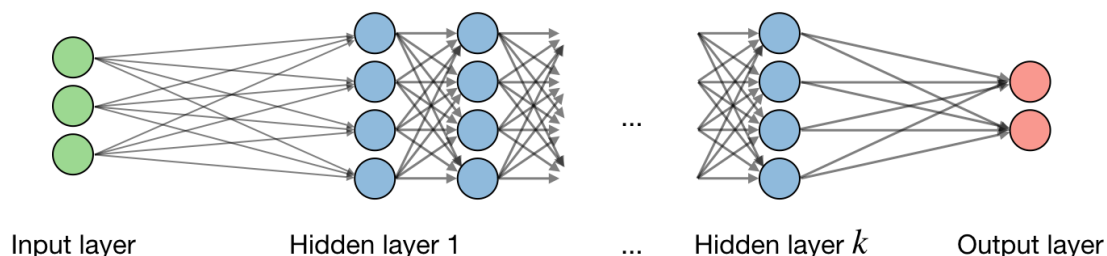
Kolejnym takim wynalazkiem, który łączy te dwa aspekty jest sztuczna inteligencja. Już od dawien dawna, przeróżne filmy czy książki science-fiction rozbudzały naszą wyobraźnię, o istnieniu robotów, sztucznej inteligencji równej ludzkiej, czy nawet przewyższającej ją.

Inspiracją w pierwszych krokach, ku stowrzeniu sztucznej inteligencji, była ludzka sieć neuronowa, to jak działa nasz mózg. Taką ideę, starano się przekuć w pierwsze wzory, pierwsze sieci, załączki sztucznej inteligencji. Początkowo był to prosty matematyczny opis komórki neuronowej przez McCullocha i Pittsa w 1943 [2]. W połączeniu z zagadnieniem przetwarzania danych mógl modelować proste funkcje logiczne. Dopiero w 1949 roku Hebb sformułował regułę, którą uznaje się za pierwszą regułę uczenia sztucznych sieci neuronowych[2].

Obecnie sieci i modeli sieci jest tysiące. Do najpopularniejszych należą np CNN - Convolutional Neural Network, czy RNN - Recurrent Neural Network. W tym projekcie wykorzystujemy sieci typu GAN, czyli Generative Adversarial Network, o której więcej w kolejnym podrozdziale.

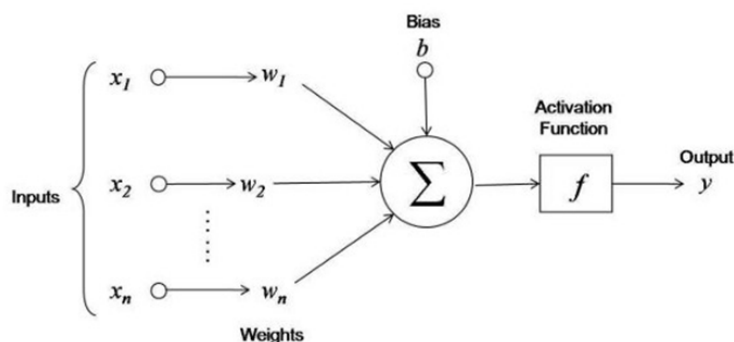
2.2 Budowa sieci neuronowej

Najmniejszą składową sieci neuronowej są oczywiście neurony, te neurony są poukładane po kilka w tak zwane warstwy. Najmniejsza ilość warstw to 3. Składają się z warstwy wejściowej, reprezentuje dane wejściowe w postaci numerycznej, warstwy ukryte, liczba mnoga tu jest celowa, ponieważ ich może wystąpić nieskończoność, to one wykonują obliczenia. Ostatnim typem warstwy jest warstwa wyjściowa, ona generuje dane wyjściowe.



Rysunek 1: Budowa sieci neuronowej [3]

Bardzo często wskazane, niekiedy wymagane, jest przekształcenie danych. Robi się ją, w zależności od potrzeb, zbioru danych i tego czego oczekujemy i chcemy uzyskać. Naturalnie jeśli zbiorem danych jest zbiór zdjęć, czy tekst to wymagane jest przekształcenie na postać liczbową, ale często wymagane jest dodatkowe przekształcenie jak normalizacja czy standaryzacja. Powodem przekształcenia może być fakt iż, większość algorytmów zakłada, że wszystkie cechy mają porównywalny zakres wartości. Takie informacje jak różne jednostki miary, ceny czy temperatury nie są zrozumiałe dla algorytmów, a mogą mieć wartości o różnicy kilku rzędów wielkości. W tym celu dane muszą być przekształcone. Zajmijmy się teraz najmniejszą częścią sieci, czyli neuronem.



Rysunek 2: Budowa neuronu [3]

”W neuronie wartości sygnałów wejściowych (oznaczone na Rysunku jako x_1 , x_2 , ...

, x_n) mnożone są przez wagi (oznaczone jako $w_1, w_2, \dots, w_n$), a iloczyny są ze sobą dodawane (do wyniku dodawana jest wartość b , która nie jest mnożona przez wartość sygnału wejściowego). Wynik rachunków poddawany jest funkcji aktywacji, która decyduje o ostatecznej wartości wysyłanego sygnału. W najprostszym przypadku może to być funkcja progowa (1 dla wartości dodatnich, 0 dla wartości niedodatnich), która przypomina działanie ludzkiego neuronu: „wysyłam sygnał lub nie”. [3] Sieć może składać się z nieskończonej ilości neuronów, jak również warstw. Do każdego neuronu przekazywana jest informacja, wykonuje on obliczenia, w dalszej kolejności wysyła sygnał do kolejnej warstwy. Następnie, cykl się powtarza i trwa dalej, aż do ostatniej, warstwy wyjściowej.

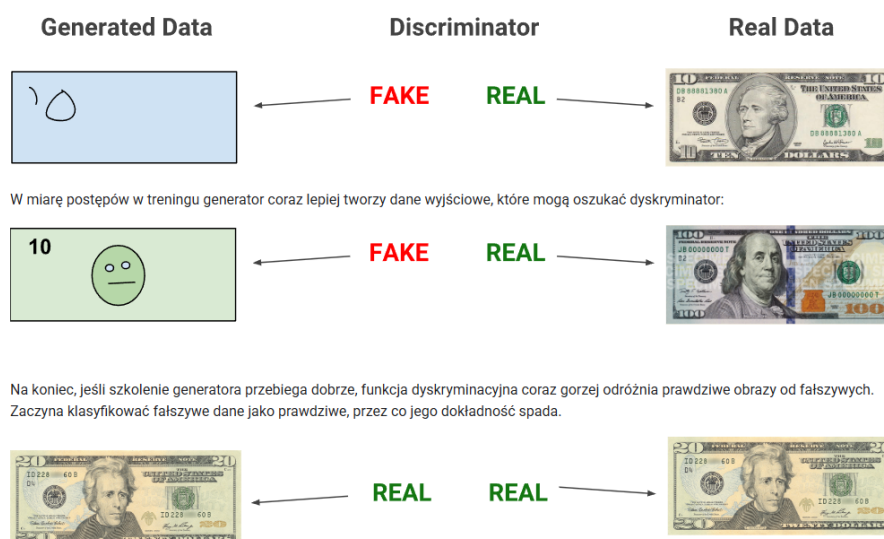
2.3 Pojęcia związane z sieciami neuronowymi

- Overfitting - jest to zjawisko gdy model jest zbyt dobrze dopasowany do zbioru treningowego, a nie radzi sobie z danymi testowymi (słaba zdolność do generalizacji nowych danych) [4]
- Epoka - jedno przejście sieci przez wszystkie dane treningowe
- Learning rate - jak szybko model zmienia i dopasowuje wagi parametrów i ”uczy się” z poprzednich iteracji
- Batch size - ile danych model przetworzy podczas każdej iteracji treningu, zanim zmieni wartości parametrów
- Lambda - parametr regularyzacji, ma za zadanie uniknąć overfitingu i wzmocnić dokładność modelu, poprzez nakładanie kar na wagi. W tym przypadku lambda ma wpływ na stratę generatora. Domyślną wartością używaną w Pix2Pix jest 100. Przy małych wartościach np. 1-10 generator będzie bardziej skupiać się na oszukaniu dyskriminatora, wyniki wyglądają realistycznie ale nie odwzorowują dobrze obrazu docelowego. Dla dużych wartości np 100-200 generator skupia się bardziej na dokładności pixelowej, wyniki będą jak najdokładniejsze oryginałowi, ale mogą być rozmazane.

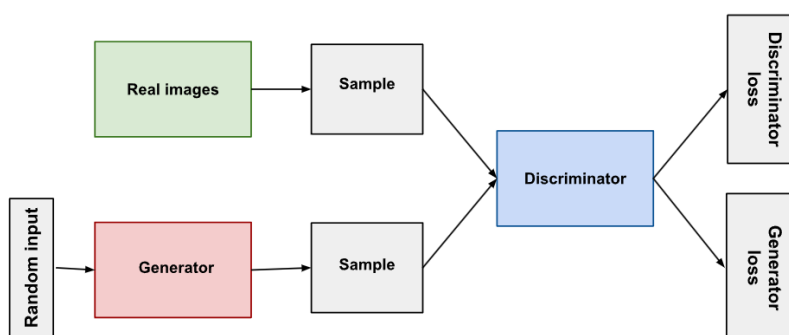
2.4 Sieci typu GAN

Sieci GAN czyli Generative Adversarial Network, a tłumacząc na polski, generatywne sieci współzawodnicze. Jak sama nazwa wskazuje są to sieci generatywne, czyli generują dane, najczęściej są to obrazy, ale nie ma ograniczeń, mogą być to również np tekst czy obraz. Sieci GAN składają się tak naprawdę z dwóch sieci, które ze sobą równocześnie współzawodniczą. Bardzo często są porównywane do fałszowania pieniędzy, mamy jedną sieć, która się uczy generować jak najdokładniejsze fałszywki, a drugą, która ma za zadanie odrzucać fałszywki. Podzielone są na:

- Generator uczy się generować wiarygodne dane i docelowo oszukać dyskryminator, że to co generuje jest prawdziwe
- Dyskryminator, uczy się odróżniać prawdę od fikcji



Rysunek 3: Generative Adversarial Network [5]



Rysunek 4: Ogólny model sieci GAN [5]

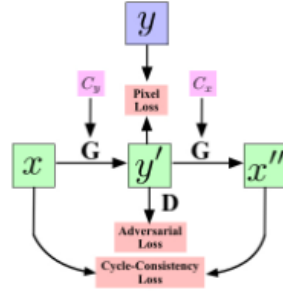
Sieci GAN mają oczywiście swoje podtypy, czyli już wcześniej stworzone wariacje tej sieci, które są powszechnie znane i używane. Wyróżnić można:

- Pix2Pix[6] jest to sieć, która przetwarza jedno zdjęcie w drugie, co jest istotne w tej sieci to to, że dane muszą być sparowane, czyli jeden z obrazów jest obrazem wejściowym, a drugim obrazem wyjściowym
- CycleGAN[7] jest bardzo podobny do Pix2Pix tylko zbiór danych nie jest sparowany. Można za jego pomocą robić choćby takie przekształcenia obrazów jak: zamiana z koni na zebry czy jabłek na pomarańcze.

Ten pierwszy został wykorzystany do realizacji niniejszej pracy. Jednakże, początkowo to ten CycleGAN był faworytem. Niestety po długich testach nie okazał się odpowiednim rozwiązaniem, o tym w oddzielnym rozdziale.

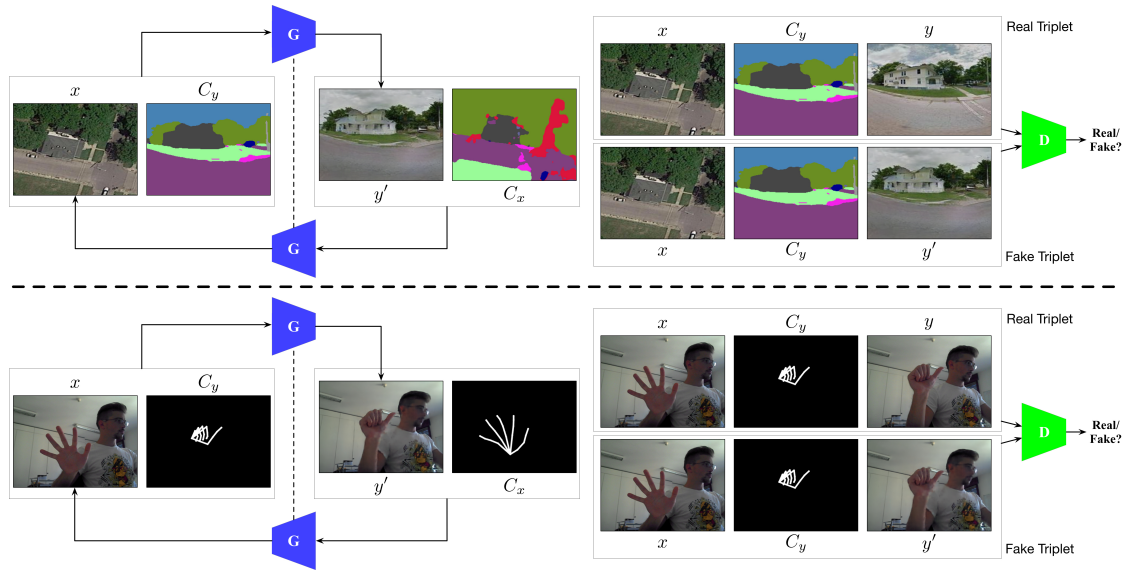
2.5 GestureGAN

GestureGAN[8] jest to propozycja stworzona przez specjalistów z różnych uczelni, takich jak Oxford czy Texas State University. W swoim założeniu ma robić to samo co sieć stworzona na potrzeby niniejszej pracy, jednakże to jej budowa jest tym co je rozróżnia. Stanowiła ogromną inspirację, lecz jedyną cechą wspólną jest wykorzystywanie mediapipe do ekstrakcji szkieletu. Co je np rozróżnia to, że GestureGAN otrzymuje dwa zdjęcia na wejściu, jedno prawdziwe, drugie samego szkieletu docelowego i na wyjściu otrzymujemy wygenerowane zdjęcie i pierwotny szkielet. Takie rozwiązanie było odpowiednie w przypadku, wykorzystywanego w pracy GestureGAN, zbioru danych, ponieważ był sparowany. To znaczy ta sama osoba wykonywała wszystkie gesty. Niestety w wykorzystanym, w niniejszej pracy, zbiorze danych nie było takiej możliwości. Dane stały się sparowane nieco sztucznie, poprzez wygenerowanie szkieletów dłoni oraz maski i nałożenie ich na zdjęcie. Bardzo dużą zaletą tego projektu jest to, że jest bardzo uniwersalny, z każdego gestu jesteśmy w stanie uzyskać każdy gest.



Rysunek 5: Model GestureGAN [8]. Oznaczenia: x i y to rzeczywiste obrazy; x' i y' to wygenerowane obrazy; x'' to zrekonstruowane obrazy; C_x i C_y to kontrolowane struktury, odpowiednio x i y ; G to generator; D to dyskryminator

i tu jeszcze przykładowe efekty i możliwości GestureGAN:

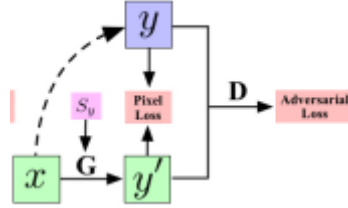


Rysunek 6: Zdjęcie wygenerowane przez GestureGAN [8]

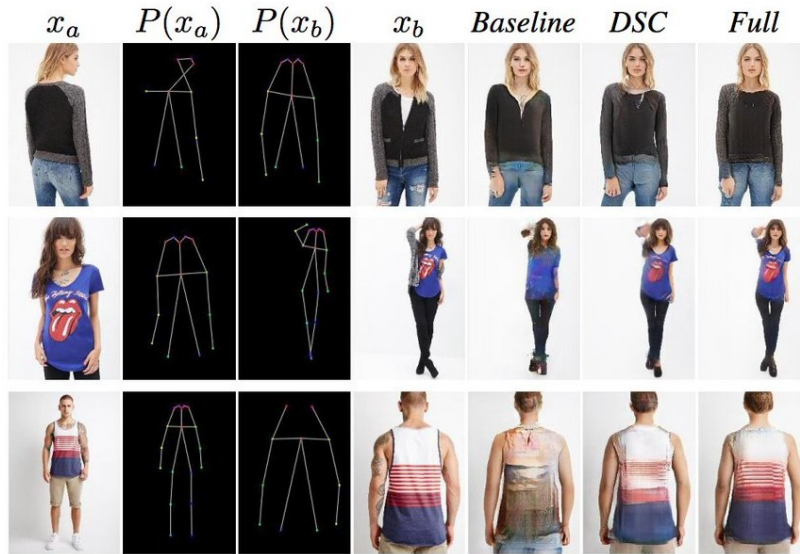
Ten projekt stanowiłby bardzo dobrą bazę, generuje wyraźnie i zróżnicowane gesty rąk, jest bardzo uniwersalny, ale problem zaczyna się już na początku. Zbiór danych, który tutaj jest wykorzystywany jest niedostępny, albo wymaga dodatkowych dostępu. Nie wszystkie dane też są wygenerowane, choćby zdjęcia szkieletów. Te dane, które były dostępne, było ich po prostu za mało. Samego projektu też ciężko zrozumieć i nie wiadomo jak z rozszerzalnością kodu. Dlatego najlepszym rozwiązaniem było przygotowanie własnego zbioru danych, własnego modelu i metodą prób i błędów, dochodzenie do rozwiązania.

2.6 PoseGAN

PoseGAN[9] jest dość zbliżony do GestureGAN, jednakże ma zasadniczą różnicę w swoim zastosowaniu. Tak jak GestureGAN, jak sama nazwa wskazuje służy do generowania gestów rąk, tak PoseGAN służy do generowania całych poz ludzkich. Także zakres jest większy jednakże przez fakt, że nie ma skupienia na dłoniach dokładność wykonywanego gestu nie ma tu znaczenia. Jednakże jest to zdecydowanie najbliższe wykonywanemu projektowi.



Rysunek 7: Model PoseGAN [9]. Notacje: x i y to rzeczywiste obrazy; y' to wygenerowany obraz; S_y to szkielet y ; G to generator; D to dyskryminator



Rysunek 8: Zdjęcie wygenerowane przez PoseGAN [9]

Projekt jest niewątpliwie bardzo imponujący jednakże skupia się na czymś zupełnie innym. Tak jak można zaobserwować na Rysunku 8 dłonie nawet nie są wyszczególnione na szkielecie, a przerabianie, dorabianie takiego elementu byłoby zbyt czasochłonne. Ciężko też ocenić czy przyniosłoby zadowalające rezultaty, przy innym zbiorze danych skupionych czysto na dłoniach. Sieć musiałaby się uczyć wszystkiego od nowa, dodatkowo wykorzystanie całego ludzkiego szkieletu jest zbędne. Nacisk jest stricte na dłonie, za

dużo informacji może wpłynąć negatywnie na jakość rezultatów . Dlatego niezbędne było stworzenie własnego rozwiązania.

3 Narzędzia i technologie wybrane do realizacji projektu

3.1 Python

Językiem programowania wykorzystanym do realizacji projektu jest Python. Jest to bardzo oczywisty wybór, ponieważ jest to najpopularniejszy język programowania do tworzenia sieci neuronowych, w dzisiejszych czasach. Przybliżmy go jednak.

Według oficjalnej dokumentacji, jest "łatwym do nauczenia się i potężnym językiem programowania. Posiada wydajne struktury danych wysokiego poziomu oraz proste, ale skuteczne podejście do programowania obiektowego"[10]. Co niejako wyróżnia go na tle innych języków, to fakt, że jest dynamicznie typowany oraz jest językiem interpretowanym. To oznacza, że nie posiada kompilatora a interpreter, który nie kompiluje programu do pliku wykonywalnego, a kod jest wykonywany w czasie rzeczywistym. Czyni to Python językiem łatwym w testowaniu czy wykonywaniu. Również jest przenośny, co oznacza, że ten sam kod uruchomi się niezależnie od systemu operacyjnego czy urządzenia. Sam język Python jest napisany w języku C, co oznacza, że jest bardzo szybki i wydajny. Został stworzony przez Guido van Rossuma w roku 1991. Co ciekawe jego nazwa wywodzi się z starych serii skeczów grupy Monty Python's Flying Circus.

Jednakże czemu akurat to on jest najczęściej wykorzystany do AI? Odpowiedź jest tak prosta jak sam Python jest prosty. Wynika to z tego, że Python ma bardzo prostą i czytelną składnię, co pozwala developerom, czy naukowcom skupianie się na logice i samym problemie, a nie na składni[11]. Kolejnym powodem przemawiającym dlaczego to właśnie Python jest najczęściej używany, jest bardzo duża ilość bibliotek, czy to wbudowanych, czy stworzonych przez społeczność. Biblioteki takie jak NumPy, SciPy, Matplotlib, czy wykorzystane w tym projekcie PyTorch czy MediaPipe, o których będzie w kolejnych podrozdziałach. Także kolejnym i ostatnim już aspektem, o którym warto wspomnieć, jest społeczność. To dzięki dużej liczbie osób i ogromnym zebranych doświadczeniom, tworzenie dowolnego projektu staje się znacznie prostsze. Praktycznie każdy projekt, aspekt projektu czy problem, ktoś już natrafił, dzięki czemu możemy szybko i sprawnie rozwiązywać

problem.

3.2 PyTorch

”PyTorch to w pełni funkcjonalny framework do tworzenia modeli do deep learningu, który jest typem machine learningu, najczęściej wykorzystywanym w aplikacjach takich jak rozpoznawanie obrazów czy procesowanie języka.”[12] Biblioteka ta została napisana w Pythonie, przez developerów z Facebook AI Research, oraz ma doskonałe wsparcie do wykorzystywania GPU, szczególnie dla GPU od firmy Nvidia. Co czyni ją idealną biblioteką dla tego projektu. Jednak projekt dotyczy generowania obrazów rąk, czyli dobre wykorzystanie GPU jest bardzo wskazane. Początkowo jednak używana była konkurencyjna biblioteka, a dokładnie Tensorflow, jednakże była wolniejsza, miała gorsze wykorzystanie GPU i była mniej klarowna od PyTorch, co zaważyło na finalnym wyborze.

3.3 MediaPipe

”MediaPipe Solutions to zestaw bibliotek i narzędzi, które umożliwiają szybkie stosowanie w aplikacjach technik sztucznej inteligencji (AI) i uczenia maszynowego (ML).”[13] Opis ten pochodzi z oficjalnej dokumentacji Mediapipe, jednakże sam opis jest zbyt ogólny. Ta biblioteka ma masę możliwości i zastosowań. Do nich można zaliczyć np. rozpoznawanie obrazu. Nie chcemy pisać sieci do rozpoznawania, czy na danym obrazku jest pies czy kot? Mediapipe daje gotowe rozwiązanie! Poza tym do wyboru jest też klasyfikacja obrazu, segmentacja obrazu, wykrywanie twarzy, rozpoznawanie gestu, oraz to co było niezbędne w tym projekcie to wykrywanie rąk i tworzenie szkieletu. To dzięki tej bibliotece udało się wyłuskać szkielet ręki na każdym z obrazów, ze zbioru danych, a następnie nałożyć go na zdjęcie wraz z maską i przekazane do modelu. Później była przydatna do tworzenia kolejnych klatek animacji, przechodzenia z jednego gestu, w drugi.

3.4 OpenCV

OpenCV jest to biblioteka do machine learningu oraz computer vision. Posiada w swoim arsenale dostęp do takich narzędzi jak wykrywanie i rozpoznawanie twarzy, identyfikowanie obiektów, śledzenie ruchów kamery, śledzenie obiektów 3D, usuwanie czerwonych oczu ze zdjęć, czy nawet łączenie zdjęć razem, by uzyskać obraz całej sceny o wysokiej

rozdzielczości.[14] Jednak to żadna z tych funkcji nie została użyta w projekcie. Wykrywanie i tworzenie szkieletu to zadanie Mediapipe, tworzenie modelu to działka PyTorch. OpenCV miał znacznie prostsze zadanie, został wykorzystany do ładowania zdjęć, czy to dla preprocessingu, czy już bezpośrednio do modelu. Dalej zapisywał każdy kolejny obraz w zależności od epoki, dzięki czemu można było śledzić poczynania i na koniec sklejał wszystkie klatki i tworzył z nich animację.

4 Proces tworzenia projektu

4.1 Dobór zbioru danych

Pierwszym, a zarazem niezmiernie ważnym krokiem jest wybór zbioru danych. Zbiór danych to podstawa projektu, jakość modelu ma bezpośrednie przełożenie, w zależności od danych, jak również ich przygotowania. To od niego będzie zależeć zarówno to jak długo sieć będzie się uczyć, ale przede wszystkim jakie osiągnie rezultaty. Gdy zbiór zdjęć jest wyraźny, najlepiej dane są sparowane, to sieć bardzo szybko zrozumie na czym polega różnica i na czym powinna się skupiać. Zbiór możemy nazwać sparowanym gdy każdy element z jednej grupy odpowiada dokładnie jednemu elementowi z drugiej grupy. Czyli dane są powiązane parami. W przypadku gestów zbiór sparowany, to taki gdzie dana osoba wykonała wszystkie gesty, w tych samych warunkach.

Poszukiwania były zaskakująco problematyczne, głównie ze względu na słabą współpracę z biblioteką Mediapipe. Często zdjęcia z innych zbiorów, to były białe plamy na czarnym tle, przez co ciężko było nawet rozróżnić ustawienie dłoni czy choćby palce. Inny dataset prezentował zdjęcie robione kamerą nocną, co w zasadzie prowadziło do takich samych rezultatów, jak w poprzednio wspomnianym zbiorze. Mediapipe nie umiał wychwycić gdzie znajduje się dłoń, palce czy stawy.

Kolejne próby były już na zbiorze przedstawiającym zdjęcia liter przedstawianych w języku migowym. Ten zbiór był dobry, ze względu na to, że przedstawiał same dłonie, na jednolitym tle, był sparowany. Jednakże co skreśliło go, to fakt, że tych zdjęć było po prostu za mało. Dla problemu takiego jak generowanie gestów, tych zdjęć musi być dużo. Tam było ich mniej więcej 30 na gest. Jest to dużo za mało. Minimum stanowi około 500-1000 zdjęć na gest, dodatkowo najlepiej jakby te zdjęcia były na jednolitym tle, żeby niepotrzebnie nie utrudniać zadania sieci. Wraz z ilością zdjęć, rośnie jakość modelu,

ale też uniwersalność. Przykładowo, zmiany tła, czy oświetlenia nie powinno znacząco zmniejszyć jakości rezultatów.

Finalnie wybór padł na zbiór HaGrid [15]. Ten zbiór nie jest niestety sparowany, jednakże posiada na tyle dużo zalet, że warto było go wybrać.

Po pierwsze jest ogromny. Cały posiada aż 1.5TB, 1 086 158 zdjęć w FullHD, podzielonych na 33 klasy gestów. Co więcej, do zbioru jest dołączony plik json dla każdego gestu, poniżej przykładowy fragment dla jednego zdjęcia:

```
"04c49801-1101-4b4e-82d0-d4607cd01df0": {  
  "bboxes": [  
    [0.06944444444, 0.3104166667, 0.2666666667, 0.2640625],  
    [0.5993055556, 0.2875, 0.2569444444, 0.2760416667]  
  ],  
  "labels": [  
    "thumb_index2",  
    "thumb_index2"  
  ],  
  "united_bbox": [  
    [0.06944444444, 0.2875, 0.7868055556, 0.2869791667]  
  ],  
  "united_label": [  
    "thumb_index2"  
  ],  
  "user_id": "2fe6a9156ff8ca27fbce8ada318c592b",  
  "hand_landmarks": [  
    [  
      [0.37233507701702123, 0.5935673528948108],  
      [0.3997604810145188, 0.5925499847441514],  
      ...  
    ],  
    [  
      [0.37388438145820907, 0.47547576284667353],  
      [0.39460467775730607, 0.4698847093520443],  
      ...  
    ]  
  ]  
}
```

```
]
"meta": {
  "age": [24.41],
  "gender": ["female"],
  "race": ["White"]
}
```

Jak widzimy znajduje się tu bardzo dużo przydatnych informacji. Do najważniejszych należą:

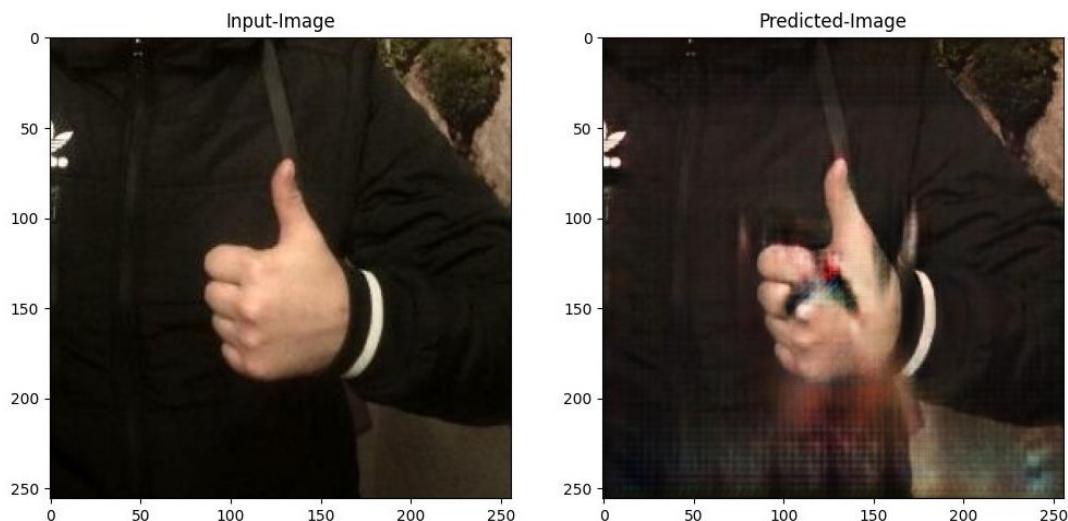
- bboxes - są to koordynaty na jakim obszarze znajdują się dłonie, przydało się to do przycinania zdjęć do wielkości 256x256, która jest standardem dla sieci GAN
- labels - dzięki temu wiadomo było którą dłoń wykorzystać do analizy czy też przetwarzania obrazu
- hand landmarks - to koordynaty położenia palca, stawów, oraz dłoni

Tak jak można zauważyć jest to doskonały zbiór danych, do ideału brakuje jednego, żeby był sparowany, ale i to nie jest problem, o czym więcej w rozdziale o preprocessingu.

4.2 Wybór architektury sieci

Wybór architektury był nieco uproszczony, bo z założenia projekt wymaga użycia sieci GAN, jednakże podtypów sieci jest dużo. Początkowo wybrany został CycleGAN, w związku z tym, że wybrany zbiór nie był sparowany, a CycleGAN dzięki dodatkowej funkcji straty, wykorzystuje utratę spójności cyklu żeby trening nie musiał się odbywać na sparowanych danych. Jako bazę posłużył tutorial z oficjalnej dokumentacji Tensorflow [16]. Z założenia wszystko wydawało się że powinno zadziałać. Na tutorialu robione są z koni zebry, czy z jabłek pomarańcze. To w zasadzie czemu miałyby sobie nie poradzić z gestami?

Rzeczywistość okazała się być bardzo brutalna.



Rysunek 9: Zdjęcie wygenerowane przez CycleGAN, po 45 epokach [16]

Na pozór wydawać by się mogło, że to zdjęcie jest dobre i sieć dosyć wiernie odtworzyła rękę, jedynie pojawiają się artefakty. Zgadza się, ale nie takie było jej zadanie. Powinna nałożyć nowy gest. Ponownie wydawać by się mogło, że zaczyna zauważać na czym polega jej zadanie, na czym powinna się skupić, bo artefakty pojawiają się na dłoni. Tylko niestety przy przeróżnych konfiguracjach parametrów, ilości puszczanych epok, czy zmian, nie udało się zmusić sieć do manipulacji gestami. Z tego powodu trzeba było coś zmienić, najwyraźniej zbiór jest zbyt zróżnicowany, tła są inne, przedstawiają innych ludzi itd, żeby CycleGAN sobie poradził.

Więc wybrane zostało inne podejście. Nastąpiła zmiana architektury na Pix2Pix i zmanipulowany został zbiór żeby uczynić go sparowanym. Dokładnie co zostało zrobione jest w kolejnym rozdziale. W dużym skrócie, na rękę z danym gestem, została, na dłoń, nakładana maska i na nią szkielet gestu. Co czyni nasz zbiór sparowanym i idealnym kandydatem dla Pix2Pix.

4.3 Preprocesowanie danych

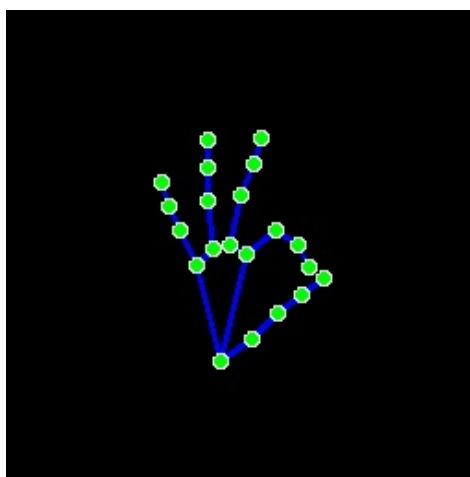
Preprocesowanie zaczęło się od najprostszej, a zarazem bardzo ważnej części. Przycięcia zdjęć do wielkości 256x256, czyli standardu dla sieci GAN.



Rysunek 10: Przykłady przed i po przycięciu

Takie zdjęcia stanowiły początkowy input i zdjęcie docelowe dla sieci. Jednakże jak zostało opisane w podrozdziale o wyborze architektury, to było za mało.

Następnie idąc za przykładem z GestureGAN [8], kolejną próbą było przekazywanie sieci zarówno zdjęcia oryginalnego, jak i oddzielnie zdjęcia docelowego gestu, ale w formie szkieletu na czarnym tle.



Rysunek 11: Wygenerowane zdjęcie szkieletu dłoni

Kolejnymi próbami obróbki zdjęć było usuwanie tła, a w zasadzie rozmazywanie go, przez wycięcie za pomocą biblioteki rembg [17]. Która to pozwala na wykrycie i usunięcie tła z obrazu. Dłoń i część, która powinna zostać jest wykrywana przy wykorzystaniu

MediaPipe.

Tu przykład po takiej obróbce:



Rysunek 12: Przykłady po obróbce tła

Jak widzimy na obrazkach powyżej (Rysunek 12), efekt jest bardzo różny. Ciężko było o jednoznaczne wykrywanie ręki. Raz zostawała sama ręka, raz wraz z człowiekiem, jeszcze innym razem potrafił zostać sam człowiek. Efekty były mało zadowalające, jednakże, mimo wszystko takie dane również zostały wypróbowane, ponownie nie przynosząc żadnych rezultatów.

Pomimo licznych niepowodzeń i ślepych zaułków, narodził się jeszcze jeden, finalny pomysł. Stworzenie zbioru danych sparowanego, w nieco sztuczny sposób. Każde zdjęcie dłoni zostało przepuszczone przez OpenCV i MediaPipe, w celu wykrycia, nałożenia maski a następnie szkieletu gestu. Dając następujący przykład:



Rysunek 13: Przykład zdjęcia z nałożoną maską i szkieletem gestu

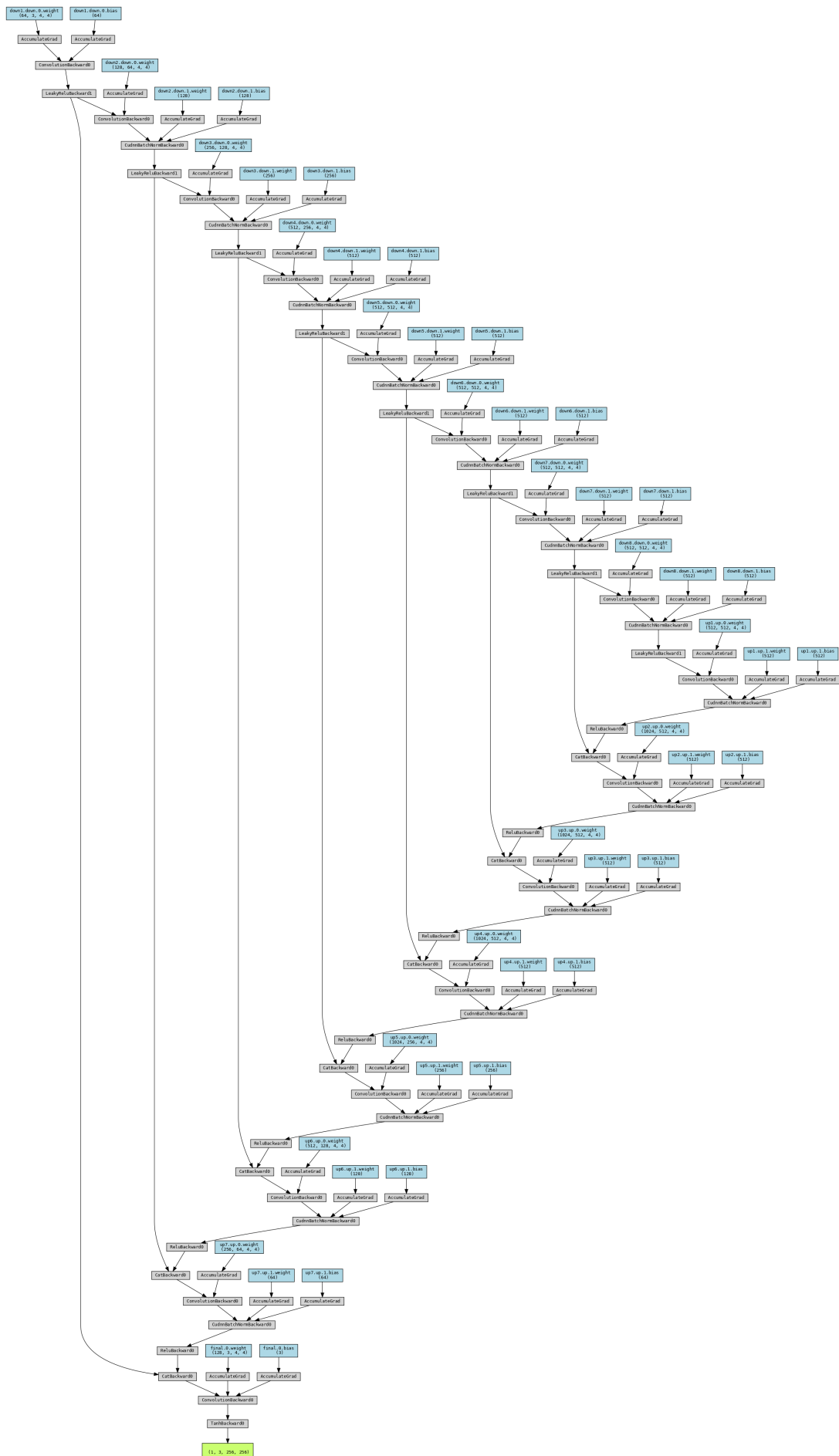
Funkcja nakładająca maski jest na tyle uniwersalna, że może przyjąć dowolny kolor i

możemy przekazać jej jako parametr, jak duży obszar ma zająć. Jednakże każde zdjęcie jest różne dlatego niezbędne było dobranie jednego rozmiaru maski, która będzie pasować dla znacznej większości obrazów. Tak przygotowane zdjęcia, a w zasadzie zbiór danych posłużył do treningu i testowania sieci.

4.4 Model sieci

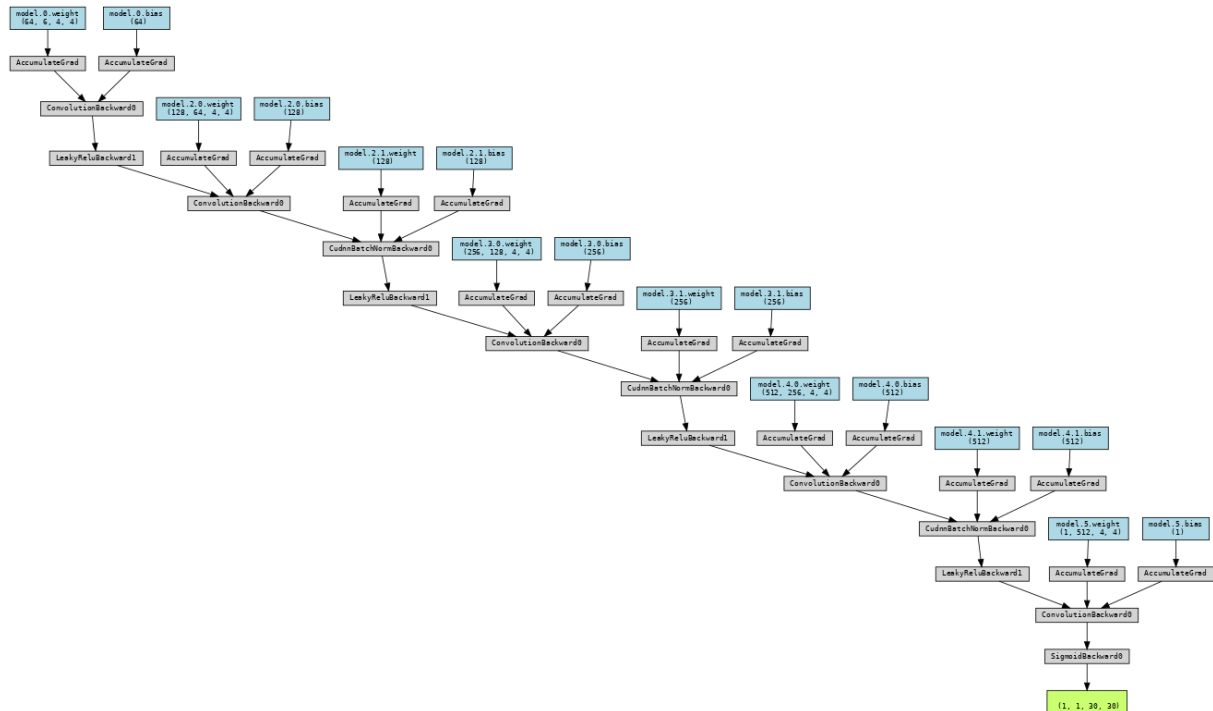
Architektura Pix2Pix jest typem warunkowej, generatywnej sieci adwersarza (cGAN), ma za zadanie nauczyć się mapowania pomiędzy obrazami, przy sparowanych danych. Pix2Pix jest bardzo uniwersalnym narzędziem, można go zastosować przykładowo do generowania kolorowych zdjęć z czarno białych, przekształcania szkiców w zdjęcia [18], czy robienia z dnia nocy. Architektura, w swojej bazowej formie składa się z generatora, opartego na U-Net, oraz discriminatora, opartego na PatchGAN.

U-Net jest siecią, w głównej mierze stworzoną do segmentacji obrazów, czyli przypisywania klasy do pikseli na obrazku, w celu np poznania kształtu danego obiektu [19]. Przed jej powstaniem klasyfikacja dotyczyła jedynie całego obrazka, ale dopiero dopiero Ronneberger, w 2015, zaproponował U-Net i to popchnęło dalej cały rozwój translacji image to image. Składa się z dwóch ścieżek, które razem składają się na kształt litery "U", stąd nazwa. Pierwsza z nich, to enkoder, jest zbliżona swoją budową do klasycznych sieci konwolucyjnych i odpowiada za klasyfikację. Zbudowany jest z bloków konwolucji, normalizacji wsadowej i LeakyRelu. Druga z kolei to dekodery [20]. Zbudowany jest z konwolucji transponowanej, normalizacji wsadowej, odrzucenia i LeakyRelu.



Rysunek 14: Generator U-Net

Dyskryminator ma bardzo proste zadanie i założenie, ma po prostu sklasyfikować czy każda łątka(patch) jest prawdziwa czy nie.[18]



Rysunek 15: Dyskryminator PatchGAN

Zaproponowane rozwiązanie nie różni się znacznie od klasycznego Pix2Pix. Jednakże co go wyróżnia i dało kolosalną różnicę w wyglądzie i jakości to zastosowanie straty percepcyjnej VGG [21].

Jest to stosunkowo nowo zaproponowane podejście. Jak dotąd sprawdzało się straty pomiędzy pojedynczymi pixelami, pomiędzy przewidywaną wartością, a faktyczną wartością dla niego, na docelowym obrazku. W tym, porównuje się ich reprezentacje cechowe uzyskane z warstw konwolucyjnych sieci VGG, wytrenowanej na ImageNet. Co daje znacznie bardziej realistyczne rezultaty. Wzór na stratę VGG to:

$$\mathcal{L}_{\text{perc}} = \sum_l \lambda_l \|\phi_l(I_{\text{gen}}) - \phi_l(I_{\text{ref}})\|_2^2$$

Jako optymalizator dla generatora i dyskryminatora został wybrany algorytm Adam. Jest to bardzo popularny algorytm optymalizacji, który jest bardzo szybki i wydajny. Poprzez połączenie zalet technik Momentum i RMSprop pozwala na dostosowywanie learning rate-u podczas trwania treningu. Dodatkowo doskonale sprawdza się w przypadku dużych zbiorów danych i złożonych modeli, ponieważ doskonale zarządza pamięcią i automatycznie dostosowuje learning rate dla każdego parametru.

4.5 Ścieżka uczenia sieci i animacja

Ścieżka uczenia sieci jest następujący:

1. Przycinane są zdjęcia pobrane z datasetu [15]
2. Generowane są obrazy z nałożoną maską i szkieletem ręki
3. Zdjęcia są ładowane, normalizowane i przekazywane do sieci
4. Sieć jest inicjalizowana, jeśli istnieją to ładowane są informacje z poprzednich uruchomień.
 - Generator
 - Dyskryminator
 - Optymalizatory generatora i dyskryminatora
 - Wartości straty dla generatora i dyskryminatora
5. Generator generuje obraz
6. Dyskryminator najpierw odbiera prawdziwy obraz i fałszywy
7. Obliczana jest strata dyskryminatora
8. Propagacja wsteczna straty dyskryminatora
9. Obliczana jest strata generatora
10. Obliczana jest strata VGG
11. Wyliczana jest całościowa strata generatora

$$g_loss_total = g_loss + self.lambdas_vgg * loss_vgg$$

12. Propagacja wsteczna straty generatora
13. Sumowana jest strata generatora i dyskryminatora dla każdej kolejnej pary zdjęć
14. Wyliczana jest średnia strata generatora i dyskryminatora, dla epoki
15. Generowane są przykładowe obrazki i robiony jest checkpoint

Animowanie przebiega następująco:

1. Brany jest przykładowy, pasujący położeniem gest docelowy i wraz z maską nakładany na obrazek, który ma być przerobiony
2. Generowane są zdjęcia, z interpolacją szkieletu z gestu startowego do docelowego.
3. Zdjęcia są przekazywane do wyuczonej sieci i zapisywane
4. Zdjęcia są sklejane w animacje dzięki OpenCV

4.6 Animacja

Animacja, jak zostało po krótkce opisano w podrozdziale 4.5, jest bardzo prosta. Zdjęcie startowe, z gestem od którego ma zacząć, oraz zdjęcie docelowe jest zaczytywane przez bibliotekę OpenCV. Następnie mediapipe, procesuje oba zdjęcia i czytuje z nich szkielety rąk. Te dane są zamieniane na tablicę numpy, ponieważ mediapipe oryginalnie zwraca obiekt specyficzny dla biblioteki, przez co nie tak łatwo operować na nim za pomocą działań matematycznych. Kolejno, współrzędne stawów są interpolowane w taki sposób żeby uzyskać kolejne klatki animacji, czyli są przesuwane o taką odległość, żeby ta zmiana zmieściła się w wyznaczonej ilości klatek. Tak uzyskane klatki są zapisywane w formacie jpg, a następnie łączone są w animacje za pomocą biblioteki OpenCV.

5 Wyniki i dyskusja

Jak zostało omówione w rozdziale wyżej, ścieżka rozwoju i dochodzenia do właściwego rozwiązania, była dosyć wyboista i trudna. W tym rozdziale zostanie przybliżone, nieco bardziej, jak wyglądały rezultaty niektórych prób. Niestety nie wszystkie zostały zapisane, ponieważ najczęściej wyglądały bardzo zbliżenie do poprzednich przez co, miało się to z celem, aby pokazywać dokładnie takie same zdjęcia, czy też suche liczby.

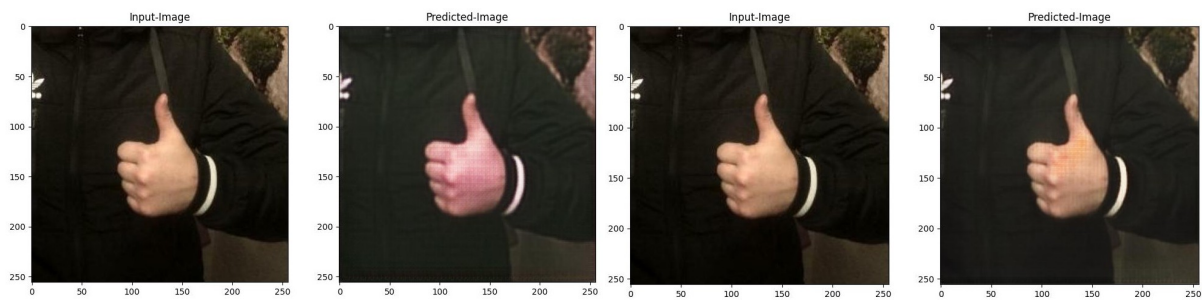
5.1 CycleGAN

Tak jak zostało omówione w podrozdziale 4.2, pierwszym wyborem jeśli chodzi o architekturę, był CycleGAN. Na pierwszy rzut oka, spełniał wszystkie wymagania projektowe. Operował na zbiorach niesparowanych, był doskonały do translacji image to image, nawet patrząc na tutorial [16] wyglądało to obiecująco. W końcu, zrobienie z koni zebry,

czy też z jablek pomarańcze, to manipulacja zdjęciem, ale to dalej to samo zdjęcie tylko z wprowadzonymi wyuczonymi zmianami. O to dokładnie chodzi, żeby dłoń ze zdjęcia, z określonym gestem, zmieniła tylko gest. Tło, postura, kolor skóry, ubranie, wszystko pozostaje takie samo.

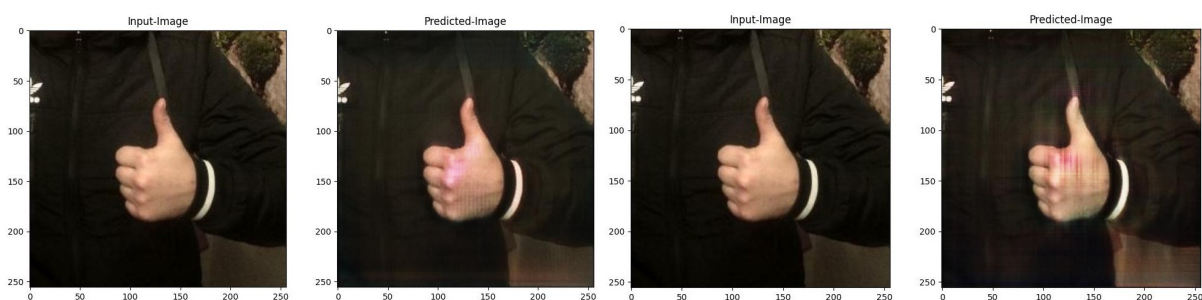
Niestety najprawdopodobniej to ta różnica, że tu manipulowana jest cecha, a w tych przypadkach kolor czy tekstura obiektu, a w tym przypadku całe ułożenie dłoni, mogło zaważyć na niepowodzeniu. Przykładowy rezultat treningu przy wykorzystaniu sieci CycleGAN: Parametry treningu:

- Liczba epok: 50
- Batch size: 1
- Buffer size: 500
- Lambda: 20



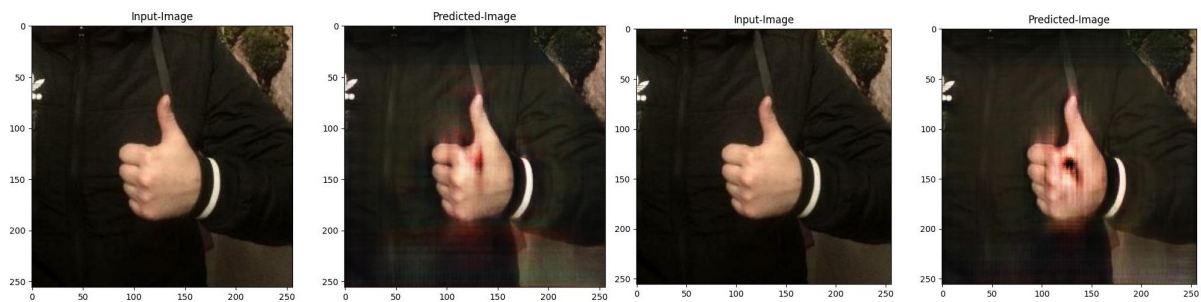
(a) Epoka 1

(b) Epoka 5



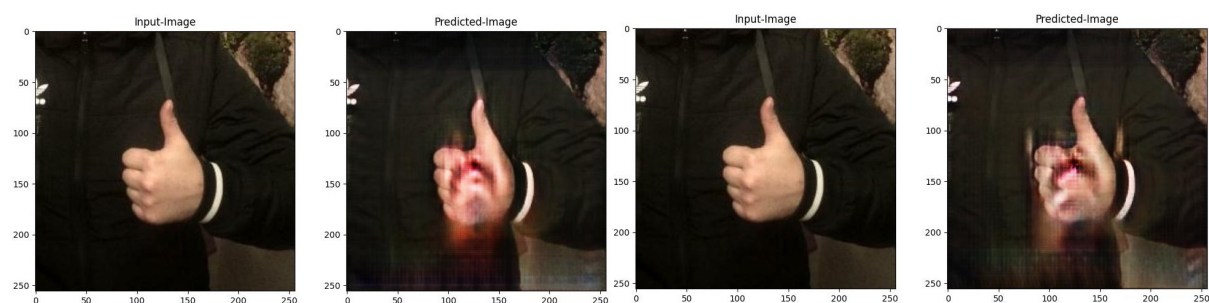
(a) Epoka 15

(b) Epoka 20



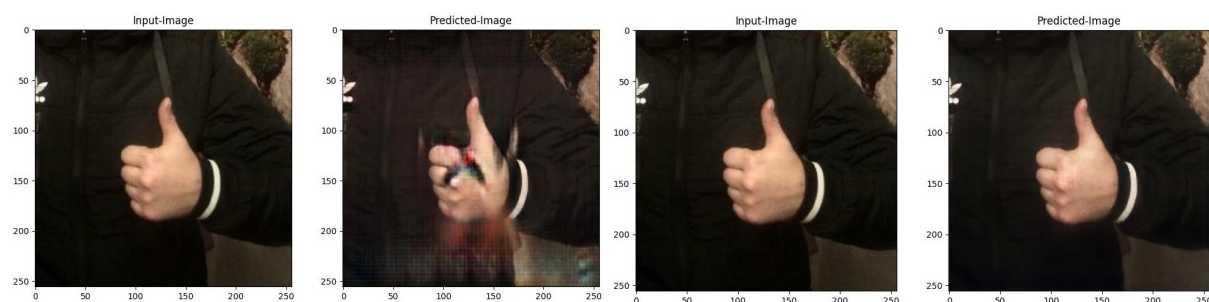
(a) Epoka 25

(b) Epoka 30



(a) Epoka 35

(b) Epoka 40

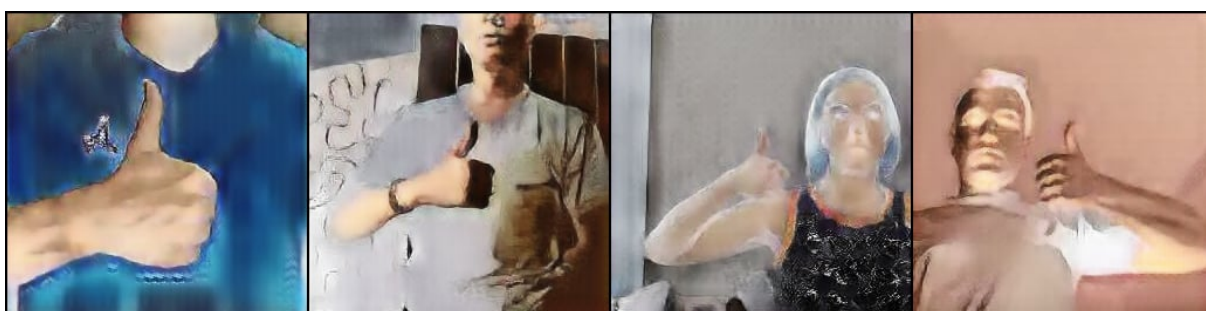
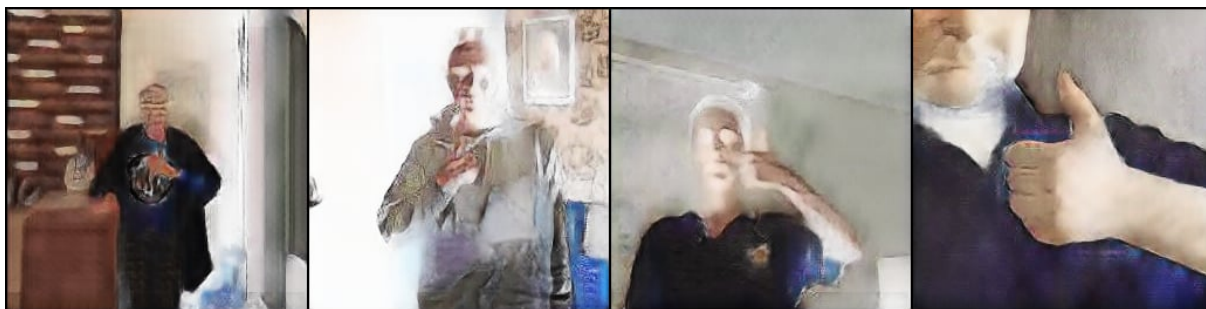


(a) Epoka 45

(b) Epoka 50

Jak możemy zaobserwować po kolejnych epokach efekt jest żaden. Widać już przy epoce 15, że jakby sieć zaczynała łapać o co chodzi, jednakże aż do epoki 45 jedyną zmianą było dalsze zniekształcenie dłoni. Co ciekawe, jak można zauważyć, w epoce 50 zdjęcie wróciło do oryginalnego wyglądu. Niestety dalsze douczania nie dawały żadnych rezultatów. Nie wspominając o jakichś zadowalających czy obiecujących.

Próby modyfikacji parametrów, zmiany w preprocessingu zbioru danych, również nijak nie wpłynęły na efekt. Dalej cel był daleko. Oto kilka przykładowych rezultatów z różnych prób:



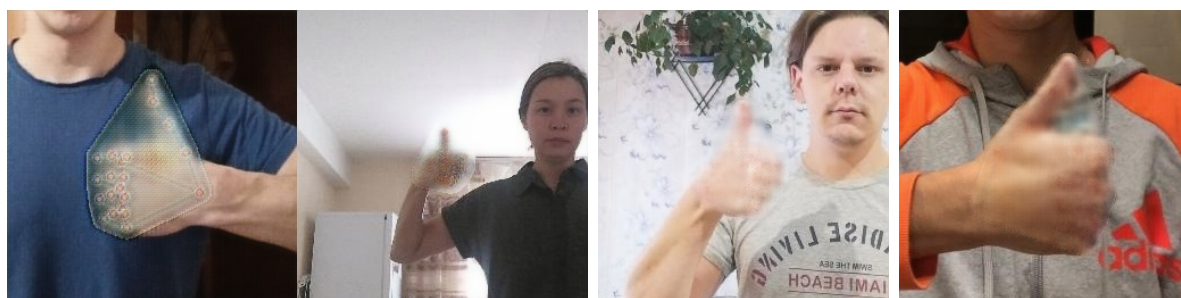
Rysunek 21: Uczenie na wyciętych dłoniach, na czarnym tle

Jak można zaobserwować na obrazkach powyżej wyniki są przeróżne. Przez zdjęcia wyglądające jak w negatywie, po rozmazania, plamy niczym z obrazu ekspresjonistycznego.

5.2 Pix2Pix

Jak zostało wspomniane w poprzedniej Sekcji 4, finalny wybór padł na Pix2Pix. Wymagało to zmiany w zbiorze danych, o czym zostało wspomniane w rozdziale o preprocesingu danych 4.3. Dopiero w końcu dzięki temu pojawiło się światło w tunelu. Początkowo nauka przebiegała tylko na zbiorze z jednym gestem. Powodem tego było sprawdzenie czy faktycznie przyniesie to jakieś rezultaty. Jednak nauka przebiegała na zdjęciu szkieletu ręki i oryginalnym zdjęciu, więc zaledwie zmiana szkieletu może wystarczyć. Okazało się ty być częściowo prawdą, ale dopiero później, sama nauka przebiegała bez problemów. Zdjęcia w tej sekcji mogą przedstawiać różne osoby, w zależności od epoki. Wynika to ze sposobu zapisywania próbek zdjęć, który bierz losowe zdjęcie przy każdym zapisie. Pozwala to na lepszy obraz tego, jak sieć radzi sobie przy różnych osobach, tłach, oświetleniach itd.

Średni czas dla epoki, przy batchu 16 wynosi około 1 minuta 53 sekundy. Czyli dla 100 epok wynik ten to ponad 3 godziny. Parametry jakie może przyjąć sieć to batch size, liczba epok, learning rate, lambda dla straty generatora, lambda dla straty VGG oraz co ile epok ma występować zapis próbek oraz punktów kontrolnych. Przykłady:



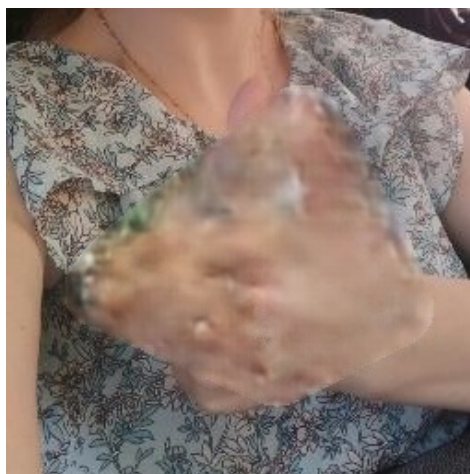
Rysunek 22: Odpowiednio: epoka 1, 20, 40, 50. Parametry: batch - 16, epochs - 50, learning rate - $1e-4$, lambda - 100

Wyniki są dalej nieco rozmazane, ale dalsze douczenie przyniosło oczekiwane rezultaty.



Rysunek 23: Efekty dalszego douczania. Odpowiednio: epoka 1, 20, 40, 60, 80, 100.

Kolejnym etapem było przekazanie zdjęcia ze zmienionym gestem, na docelowy. Niestety tu wyszedł problem z takim podejściem. Faktycznie wynik był zdecydowanie bliższy do oczekiwanego rezultatu, ale absolutnie nie był czytelny. Ciężko było rozpoznać czy to w ogóle ręka, czy jakiś potwór jak z filmu "The Thing" Johna Carpentera.



Rysunek 24: Rezultat przepuszczenia przez sieć zdjęcia z nałożonym szkieletem docelowego gestu

Co okazało się znacząco poprawić rezultaty, było dołożenie do nauki zbioru z drugim gestem, który jest docelowym gestem. Oczywiście dalej nie były zdjęcia dwóch gestów,

wykonywanych przez tą samą osobę, ale dało się rozpoznać chociaż zarysy gestu.



Rysunek 25: Rezultat po dodaniu zbioru z drugim gestem

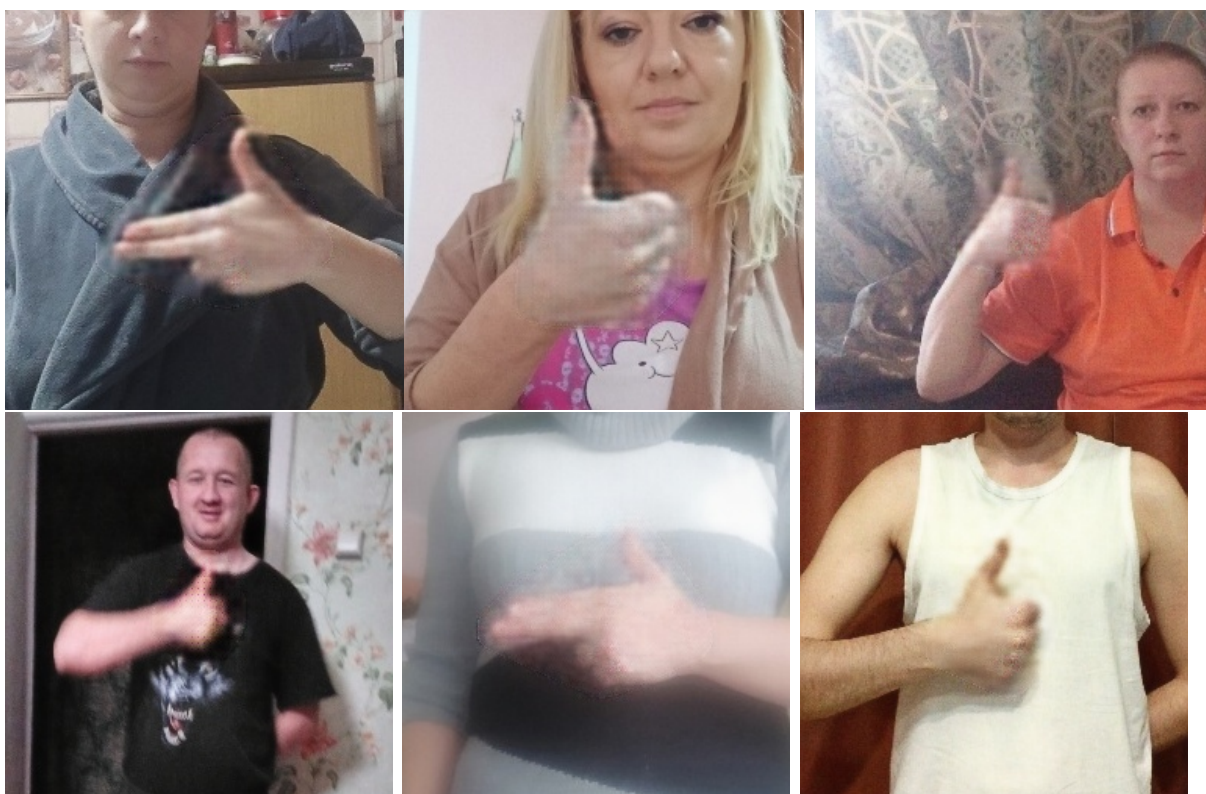
Kolejnym etapem było dodanie VGG perceptual loss, który został nieco przybliżony w rozdziale 4.4, oraz manipulacja parametrami.



Rysunek 26: Odpowiednio: epoka 1, 20, 40, 60, 80, 100. Parametry: batch - 16, epochs - 100, learning rate - $1e-4$, lambda - 10

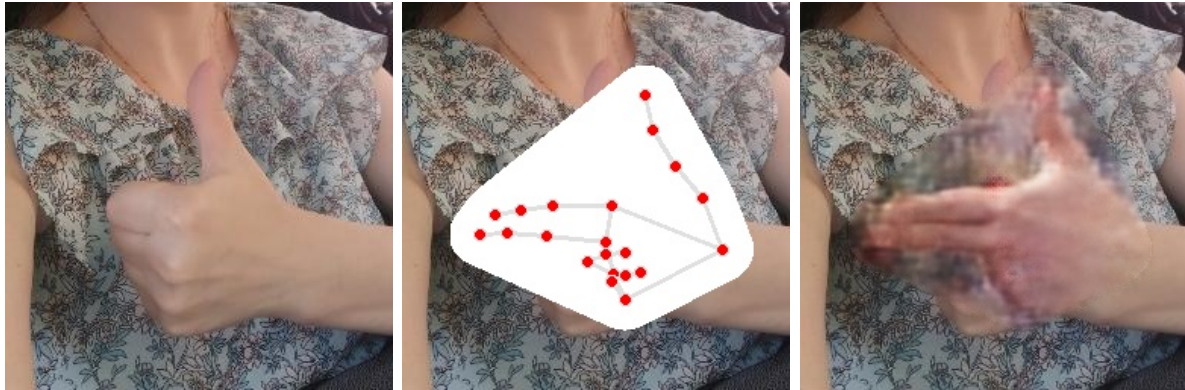


Rysunek 27: Odpowiednio: epoka 1, 20, 40, 60, 80, 100. Parametry: batch - 16, epochs - 100, learning rate - $1e-4$, lambda - 15

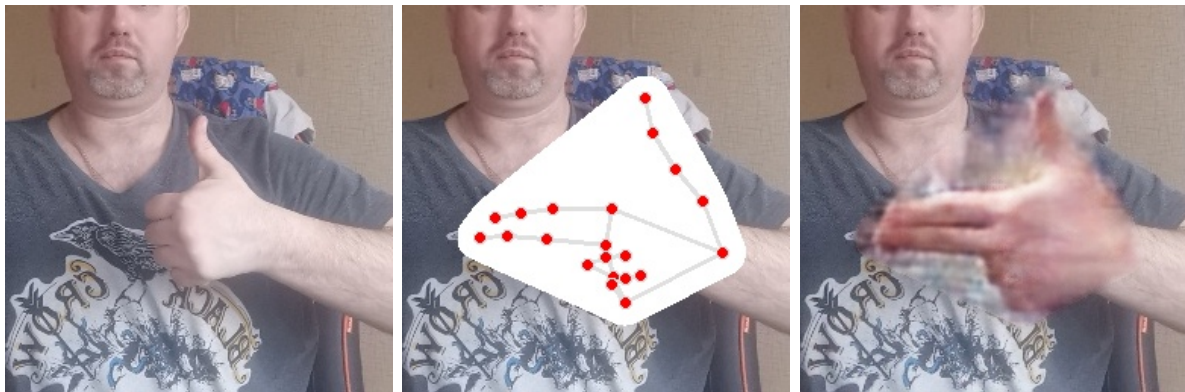


Rysunek 28: Odpowiednio: epoka 1, 20, 40, 60, 80, 100. Parametry: batch - 15, epochs - 200, learning rate - $1e-4$, lambda - 20

Na pozór rezultaty są bardzo porównywalne do tych bez VGG, jednak to gdzie najbardziej widać różnicę jest przy wygenerowaniu zdjęcia z nowym gestem:



Rysunek 29: Wygenerowane zdjęcie z dodatkiem VGG



Rysunek 30: Wygenerowane zdjęcie, kolejny przykład

Tu już rezultat był bardzo zadowalający, gest jest w pełni widoczny, palce da się wyróżnić. Sama nauka sieci zajęła 100 epok, następnie została podjęta próba douczenia, o kolejne 100 epok, ale różnica była znikoma. Więc finalnie sieć przeszła 200 epok.

Specyfikacje sprzętu wykorzystanego na potrzeby projektu:

- Procesor - AMD Ryzen 5 5600 6-Core Processor (3.50 GHz)
- 16 GB RAM
- Karta graficzna - NVIDIA GeForce RTX 3070

Dla powyższej specyfikacji został zmierzony czas sieci, od załadowania modelu, do wygenerowania pełnej animacji. Wyniki przedstawiają się następująco:

- Załadowanie modelu, wraz z poprzednim punktem kontrolnym - 8.4s

- Wygenerowanie jednego zdjęcia 0.46s
- Wygenerowanie 1 sekundy animacji, czyli 30 klatek - 0.46s
- Stworzenie z nich animacji - 0.04s

Jest to bardzo dobry czas, czego w głównej mierze zasługą jest specyfikacja sprzętu. Nie jest to nowoczesny komputer, zdecydowanie bliżej mu do średniej, czy niskiej półki, ale jak najbardziej spełnił oczekiwania i sprawnie wykonał zadanie. Szczególnie ważna jest jakość sprzętu przy nauce sieci, a jeszcze bardziej, gdy ma się do czynienia ze zdjęciami. Wtedy niezbędna jest dobra karta graficzna. RTX 3070 dla 100 epok potrzebował ponad 3h do nauki, gdzie dla sprzętu niższej półki ten czas znacznie by wzrósł. Choćby ze względu na fakt, iż firma Nvidia coraz bardziej stawia na AI i rozwój sztucznej inteligencji, przez co kolejne modele kart, coraz lepiej wspierają AI. Przechodząc dalej, załadowanie modelu sieci zajęło niewiele ponad 8 sekund (wartość ta jest bardzo zmienna, raz przyjmuje mniejszą wartość np 4 sekundy, innym razem większą). Na ten czas składa się załadowanie modelu, zainicjowanie parametrów, wczytanie danych z punktu kontrolnego, z ostatniego zapisu. W skład danych wczytanych z punktu kontrolnego wchodzi dyskryminator, generator oraz ich strata i optymalizator. Czas załadowania modelu jest na pozór długi, ponieważ sam model nie jest skomplikowany. Jednakże należy pamiętać, że poza parametrami, niezbędne jest wczytanie dwóch modeli, czyli generatora i dyskryminatora. To samo w sobie wydłuża czas, a jeszcze należy te dwie sieci połączyć w jeden model Pix2Pix. Mając te rzeczy na uwadze, czas załadowania można uznać za odpowiedni. Kolejnym zmierzonym czasem jest wygenerowanie jednego obrazu oraz 30. Co ciekawe czas wygenerowania jednego zdjęcia jest taki sam jak wygenerowanie 30 klatek animacji. Nie można jednoznacznie stwierdzić z czego to wynika. Może to być spowodowane kwestią zarządzania pamięci, biblioteka OpenCV, która wczytuje obrazy, nie inicjuje się na nowo przy każdej klatce i na nowo jest wykorzystywana. Przy generowaniu jednej klatki, zasób może już zostać zwolniony. Niezależnie od różnicy czasów wygenerowania 1 czy 30 obrazów, sieć osiągnęła bardzo dobry wynik. Tu duży wpływ może mieć sprzęt, ale jak również odpowiednio dobrany model, który nie wykonuje dużej ilości obliczeń, a otrzymane efekty są bardzo dobre. Przechodząc do zmiany klatek w animację to czas nie dziwi, jest to łączenie zdjęć i zapis jako plik mp4. Taki proces nie jest skomplikowany i biblioteka OpenCV radzi sobie z nim doskonale. Co pozostaje do dopracowania, to poprawienie jakości tła, a dokładniej miejsca gdzie znajduje się maska dłoni.

Ostatnim i finalnym krokiem było wygenerowanie animacji zmiany gestu, co wymagało wygenerowania dodatkowych zdjęć, które posłużyły jako klatki animacji o czym więcej w rozdziale 4.5.

Projekt okazał się być zdecydowanie bardziej skomplikowany, niż pozornie było zakładane. Choć finalnie jest to rozbudowana wersja Pix2Pix, o dodatkowy element czyli VGG. Tym bardziej, że istnieje gotowe rozwiązanie, jednakże im dalej, tym trudności i przeszkody były przeróżne i niestabilne. Na szczęście finalnie udało się znaleźć złoty środek i sfinalizować projekt, jednocześnie uzyskując bardzo zadowalające rezultaty. Gdzie warto przypomnieć, że sztuczna inteligencja ma bardzo duże trudności z dłońmi. Tym bardziej imponujące są rezultaty.

6 Podsumowanie

Celem pracy było stworzenie sieci neuronowej typu GAN, która pozwoli na generowanie animacji przechodzenia z jednego gestu dłoni, w drugi. Docelowo w jak najlepszej jakości. Udało się zrealizować założone cele, sieć generuje dobrej jakości zdjęcia ze zmienionymi gestami rąk, a następnie za pomocą biblioteki OpenCV, zdjęcia łączone są w animację. Zastosowano podstawowe narzędzia i technologie do realizacji projektu. Sam Pix2Pix, który jest jednym z podstawowych podtypów sieci GAN, wystarczył by uzyskać zadowalające efekty, a dodanie VGG perceptual loss, oraz odpowiednia manipulacja parametrami, pozwoliły na uzyskanie wyraźnych i ładnych zdjęć.

W trakcie realizacji, nie brakowało trudności czy problemów. Pierwszym z nich był dobór zbioru danych, który wymagał większej ilości zdjęć, niż większość zbiorów oferowała. Finalnie wybór padł na HaGrid[15], który swoją ilością zdjęć, oraz dostarczonemu pliku json, z informacjami, dla każdego zdjęcia, okazał się być świetnym wyborem.

Kolejnym problemem okazał się być dobór architektury, choć nie ujawnił się on od razu. Dopiero po licznych próbach okazało się, że przy danym zbiorze danych jest najprawdopodobniej za trudny dla CycleGAN, ponieważ nie dawał żadnych rezultatów. Nie pozostało nic innego jak zmienić podejście i wraz z nim architekturę. Co okazało się być kluczowe do pchnięcia dalej prac.

Jak zostało wspomniane, efekt jest zadowalający, lecz nie idealny. Finalny rezultat można zaobserwować na Rysunku 29, 30, jak również na rysunku poniżej.



Rysunek 31: Wygenerowane klatki animacji dla 30 klatek. Odpowiednio 1, 5, 15, 25, 30 klatka.

Na Rysunku 31 można zauważyć, że gest startowy jak i końcowy są zdecydowanie najlepsze. Są wyraźnie widoczne palce wskazujący, środkowy oraz kciuk. Pozostałe palce są zwinięte w pięść, więc niestety one nie są widoczne. Fakt iż, klatki pośrednie są rozmazane, jest spowodowane tym, że to właśnie na tych dwóch gestach uczyła się sieć, a wszystkie inne wariacje ustawień szkieletu stanowią większy problem. Jak najbardziej, da się wyszczególnić zarys dłoni, ale porównując z wejściowym i wyjściowym gestem jest bardzo mało wyraźne. Najbardziej rzuca się to w oczy na samej animacji, gdzie klatki pośrednie są mało wyraźne, podobnie obszar, gdzie znajdowała się maska dłoni. Potencjalnie jest to ściśle ze sobą powiązane. Pierwszy problem udało by się rozwiązać, choćby poprzez przekazanie sieci do nauki klatek pośrednich wraz z maską dłoni i szkieletem. Niestety oryginalne zdjęcia dla tych klatek nie istnieją, czyli nie ma prawdziwego zdjęcia gdzie dana osoba wykonała gest. Można by to rozwiązać poprzez znalezienie zbioru danych z filmami zmiany gestu, a następnie pocięcie go na klatki i nałożenie maski oraz szkieletu. Niestety byłoby to bardzo czasochłonne i zasobożerne. Dlatego niezbędny by był dodatkowy czas na zbadanie potencjalnego lepszego rozwiązania.

Podsumowując, realizacja przebiegła pomyślnie, ale nie bez problemowo. Cel został zrealizowany, a samo zagadnienie było interesujące. Szczególnie przy obecnym zapotrzebowaniu na sztuczną inteligencję, niniejsza praca bardzo wpisuje się w panujące trendy i ma dużą perspektywę rozwoju.

Bibliografia

- [1] Yu CAO i Yanbin HAO Haozhuo ZHANG i Bin ZHU. “Hand1000: Generating realistic hands from text with only 1,000 images”. In: (2025).
- [2] Bohdan Macukow. “SIECI NEURONOWE, HISTORIA BADAŃ I PODSTAWOWE MODELE”. In: (). URL: <https://pages.mini.pw.edu.pl/~macukowb/wspolne/PNEiTI.pdf>.
- [3] Blog CSI Uniwersytet Im. Adama Mickiewicza w Poznaniu. *Sieci neuronowe w NLP*. 2025. URL: <https://csi.amu.edu.pl/blog-csi/sieci-neuronowe-w-nlp>.
- [4] Xue Ying. “An Overview of Overfitting and its Solutions”. In: ().
- [5] Google. *Kurs zaawansowany Machine Learning GAN*. URL: <https://developers.google.com/machine-learning/gan?hl=pl>.
- [6] Phillip Isola i Jun-Yan Zhu i Tinghui Zhou i Alexei A. Efros. “Image-to-Image Translation with Conditional Adversarial Networks”. In: (2018).
- [7] Jun-Yan Zhu i Taesung Park i Phillip Isola i Alexei A. Efros. “Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks”. In: (2020).
- [8] Hao Tang i Hong Liu i Nicu Sebe. “Unified Generative Adversarial Networks for Controllable Image-to-Image Translation”. In: (2020).
- [9] Aliaksandr Siarohin i Enver Sangineto i Stephane Lathuiliere i Nicu Sebe. “Deformable GANs for Pose-based Human Image Generationn”. In: (2018).
- [10] Python Software Foundation. *The Python Tutorial*. 2025. URL: <https://docs.python.org/3/tutorial/index.html>.
- [11] Dhruvitkumar Talati. “Python: The alchemist behind AI’s intelligent evolution”. In: (2021).
- [12] Nvidia. *PyTorch*. URL: <https://www.nvidia.com/en-us/glossary/pytorch/>.
- [13] Google. *Przewodnik po rozwiązaniach MediaPipe*. URL: <https://ai.google.dev/edge/mediapipe/solutions/guide?hl=pl>.
- [14] OpenCV. *OpenCV is the world’s biggest computer vision library*. URL: <https://opencv.org/about/>.

- [15] Anton Nuzhdin et al. *HaGRIDv2: 1M Images for Static and Dynamic Hand Gesture Recognition*. 2024. arXiv: 2412.01508 [cs.CV]. URL: <https://arxiv.org/abs/2412.01508>.
- [16] Tensorflow. *CycleGAN*. URL: <https://www.tensorflow.org/tutorials/generative/cyclegan?hl=pl>.
- [17] Daniel Gatis. *Rembg*. URL: <https://github.com/danielgatis/rembg>.
- [18] Tensorflow. *pix2pix: Tłumaczenie obrazu na obraz z warunkowym GAN*. URL: https://www.tensorflow.org/tutorials/generative/pix2pix?hl=pl#import_tensorflow_and_other_libraries.
- [19] Tensorflow. *Segmentacja obrazu*. URL: https://www.tensorflow.org/tutorials/images/segmentation?hl=pl#what_is_image_segmentation.
- [20] Jocelyn Chanussot Xin Wu Danfeng Hong. “UIU-Net: U-Net in U-Net for Infrared Small Object Detection”. In: (2022).
- [21] Li Fei-Fei Justin Johnson Alexandre Alahi. “Perceptual Losses for Real-Time Style Transfer and Super-Resolution”. In: (2016).

Spis rysunków

1	Budowa sieci neuronowej [3]	5
2	Budowa neuronu [3]	5
3	Generative Adversarial Network [5]	7
4	Ogólny model sieci GAN [5]	7
5	Model GestureGAN [8]. Oznaczenia: x i y to rzeczywiste obrazy; x' i y' to wygenerowane obrazy; x'' to zrekonstruowane obrazy; C_x i C_y to kontrolowane struktury, odpowiednio x i y ; G to generator; D to dyskryminator . . .	9
6	Zdjęcie wygenerowane przez GestureGAN [8]	9
7	Model PoseGAN [9]. Notacje: x i y to rzeczywiste obrazy; y' to wygenerowany obraz; S_y to szkielet y ; G to generator; D to dyskryminator	10
8	Zdjęcie wygenerowane przez PoseGAN [9]	10
9	Zdjęcie wygenerowane przez CycleGAN, po 45 epokach [16]	16
10	Przykłady przed i po przycięciu	17
11	Wygenerowane zdjęcie szkieletu dłoni	17
12	Przykłady po obróbce tła	18
13	Przykład zdjęcia z nałożoną maską i szkieletem gestu	18
14	Generator U-Net	20
15	Dyskryminator PatchGAN	21
21	Uczenie na wyciętych dłoniach, na czarnym tle	26
22	Odpowiednio: epoka 1, 20, 40, 50. Parametry: batch - 16, epochs - 50, learning rate - 1e-4, lambda - 100	27
23	Efekty dalszego douczania. Odpowiednio: epoka 1, 20, 40, 60, 80, 100. . . .	28
24	Rezultat przepuszczenia przez sieć zdjęcia z nałożonym szkieletem docelowego gestu	28
25	Rezultat po dodaniu zbioru z drugim gestem	29
26	Odpowiednio: epoka 1, 20, 40, 60, 80, 100. Parametry: batch - 16, epochs - 100, learning rate - 1e-4, lambda - 10	29
27	Odpowiednio: epoka 1, 20, 40, 60, 80, 100. Parametry: batch - 16, epochs - 100, learning rate - 1e-4, lambda - 15	30
28	Odpowiednio: epoka 1, 20, 40, 60, 80, 100. Parametry: batch - 15, epochs - 200, learning rate - 1e-4, lambda - 20	30

29	Wygenerowane zdjęcie z dodatkiem VGG	31
30	Wygenerowane zdjęcie, kolejny przykład	31
31	Wygenerowane klatki animacji dla 30 klatek. Odpowiednio 1, 5, 15, 25, 30 klatka.	34